

PROGETTO DI LABORATORIO DI ALGORITMI

○ Progetto 05

Iacopo Salmeri -0774955

CAPITOLO 1: FORMALIZZAZIONE DEL PROBLEMA

1.1 Introduzione al contesto reale

La gestione efficiente dell'informazione è un requisito fondamentale per i moderni sistemi informatici. In un contesto reale come quello del catalogo di una libreria, il sistema è chiamato a gestire migliaia di record testuali e numerici. Il catalogo non è un insieme statico, ma dinamico, e come tale, ha bisogno di operazioni efficienti per poter essere consultato e aggiornato (con l'inserimento e/o la cancellazione di volumi) in tempo reale.

L'obiettivo di questo problema è progettare e implementare un'infrastruttura software in grado di garantire all'utente dei tempi di risposta fortemente limitati, indipendentemente dalla mole di dati e sequenze operative richieste.

1.2 Modellazione computazionale: costruire il problema reale con un problema algoritmico

Per tradurre il problema reale in codice, l'entità '*Book*' è stata modellata tramite un'apposita struttura dati.

Il problema è stato quindi modellato come l'implementazione di una Symbol Table. Ad ogni entità è stata assegnata una coppia **Key-Value**:

- **Key**: Il codice ISBN del volume. Agisce come identificatore e come parametro di ordinamento.
- **Value**: Il pacchetto informativo. Contiene Titolo, Autore e Anno di pubblicazione.

1.3 Definizione formale del problema

Dal punto di vista matematico e computazionale, il problema è modellabile come la gestione di una mappa dinamica.

Sia K l'universo delle chiavi possibili (i codici ISBN validi) e sia V l'universo dei valori associabili (i dati del libro). Il sistema deve mantenere un insieme dinamico S , che rappresenta una relazione uno-a-uno $S \subset K \times V$. Data una chiave $k \in K$ e un valore $v \in V$, la struttura dati scelta deve supportare le seguenti operazioni minimizzando la complessità temporale asintotica:

- **INSERT(k, v)**: Inserisce l'elemento con chiave k e valore v nell'insieme S , o ne aggiorna il valore se $k \in S$.
- **SEARCH(k)**: Restituisce il valore associato alla chiave k , oppure un indicatore di fallimento se $k \notin S$.
- **DELETE(k)**: Rimuove l'elemento con chiave k dall'insieme S .
- **RANGE(k1, k2)**: Restituisce il sottoinsieme di elementi le cui chiavi sono comprese nell'intervallo chiuso $[k1, k2]$ rispettando l'ordine lessicografico.
- **PRINT**: Stampa l'intero insieme in ordine lessicografico secondo le chiavi.

1.4 Analisi dei dati

I dati in ingresso vengono forniti tramite un dataset testuale in cui ogni riga rappresenta un'operazione atomica sequenziale. Il file di input è strutturato secondo la seguente sintassi:

INSERT <ISBN> <TITOLO> <AUTORE> <ANNO>

DELETE <ISBN>

RANGE <ISBN_START> <ISBN_END>

PRINT

Esempi di flussi operativi nel file:

- *INSERT* 97807 *The Java Programming Language* Arnold 1958
- *SEARCH* 97815
- *DELETE* 97807
- *RANGE* 97800 97820
- *PRINT*

I dati in ingresso presentano caratteristiche specifiche che influenzano le scelte architetturali:

- Le chiavi sono stringhe numeriche. Il confronto logico avviene tramite **compareTo** per valutarne il peso lessicografico.
- Il flusso delle operazioni non è predicibile. I dati possono arrivare in ordine puramente casuale; nel caso peggiore, in ordine strettamente crescente o decrescente.

1.5 Possibili criticità

Nella fase di progettazione preliminare sono emerse due criticità, una di natura algoritmica e una di natura architetturale:

- **Degradazione strutturale:** Se i dati di input venissero forniti in ordine sequenziale, un banale Albero Binario di Ricerca (BST) non bilanciato degenererebbe in una lista concatenata, portando i tempi delle operazioni fondamentali da una complessità logaritmica $O(\log N)$ a una lineare $O(N)$. Questo causerebbe il collasso del sistema su input massivi.
- **Input/Output Bottleneck:** L'elaborazione di interrogazioni ampie, come la stampa intera dell'albero o la query **RANGE** su migliaia di record, rischia di saturare il buffer di scrittura del disco rigido. Questo collo di bottiglia hardware andrebbe a mascherare la reale velocità di esecuzione dell'algoritmo in memoria RAM (CPU bound), falsando le metriche di *benchmarking* e generando file di output di dimensioni ingestibili.

CAPITOLO 2: SCELTA E PROGETTAZIONE DELL'ALGORITMO

2.1 Analisi delle possibili soluzioni

Il problema richiedeva esplicitamente l'adozione di un BST. Tra le strutture dati studiate, sono state valutate tre possibili scelte:

- **Alberi Binari di Ricerca (BST Standard):** Un BST è una struttura dati dinamica usata per realizzare in modo efficiente il tipo di dato astratto Dizionario (operazioni di insert, delete e search). Nel caso medio la complessità di tempo si attesta a $O(\log N)$ per tutte le operazioni. Tuttavia, nel caso pessimo (ad esempio l'inserimento di chiavi ISBN già ordinate sequenzialmente), l'albero non prevede alcun meccanismo di bilanciamento. La complessità diventa $O(N)$ in quanto l'ABR degenera in una lista e, per trovare l'elemento più profondo,

l'algoritmo attraversa tutti i nodi uno per uno, annullando il vantaggio della struttura ad albero. Pertanto, non è una scelta applicabile a questo problema.

- **ALBERI AVL (Adelson-Velsky e Landis):** un AVL è un albero binario di ricerca "autobilanciante". Utilizza un criterio di bilanciamento basato sulle altezze dei sottoalberi, garantendo matematicamente che la differenza di altezza tra il sottoalbero destro e sinistro di qualsiasi nodo non superi mai il valore di 1. Tuttavia, la rigidità di questo bilanciamento comporta un overhead computazionale significativo durante le operazioni di inserimento e cancellazione, rendendo necessarie frequenti e complesse operazioni di rotazione.
- **ALBERI ROSSO-NERI (RB):** Un albero Rosso-Nero rappresenta un'ottima soluzione architetturale. Utilizza un criterio di bilanciamento "rilassato" basato sulla colorazione dei link. Pur permettendo all'albero di apparire visivamente asimmetrico, garantisce rigorosamente che il cammino più lungo dalla radice a una foglia vuota non sia mai più del doppio del cammino più corto. Nella variante **Left-Leaning Red-Black (L-LLRB)** proposta da Sedgwick, offre una corrispondenza biunivoca (isomorfismo) con gli alberi 2-3, realizzata utilizzando la struttura base dei BST. "Schiacciando" tutti i link rossi dell'albero, si ottiene l'albero 2-3 perfettamente bilanciato di partenza.

2.2 Motivazione della scelta

La scelta architetturale è ricaduta sull'algoritmo **Left-Leaning Red-Black Tree (L-LRB)** per i seguenti motivi tecnici:

- **Garanzia del caso pessimo:** A differenza di un BST standard, l'L-LRB garantisce che l'altezza massima dell'albero rimanga sempre proporzionale a $2 \log_2 N$. Questo assicura che le operazioni abbiano una complessità limitata asintoticamente da $O(\log N)$ anche nel peggior scenario possibile (ovvero i dati di input già ordinati).
- **Ottimizzazione del partizionamento:** Rispetto agli alberi AVL, che richiedono un bilanciamento rigido e continue rotazioni in fase di popolamento, l'L-LRB utilizza un bilanciamento "rilassato" (isomorfismo con gli alberi 2-3). Questo riduce drasticamente l'overhead computazionale durante la costruzione massiva dell'albero.
- **Semplicità implementativa:** Il vincolo "Left-Leaning" (i link rossi possono pendere solo verso sinistra) riduce simmetricamente i casi limite da gestire nel codice rispetto a un RB-Tree classico, minimizzando la possibilità di introdurre bug logici nelle rotazioni.

2.3 Strutture dati utilizzate

RedBlackBST<Key, Value>

Rappresenta l'implementazione dell'albero di ricerca. Gestisce internamente la radice (root) e coordina tutte le operazioni di navigazione e bilanciamento. L'utente può interagire con l'albero solo attraverso questa classe (principio di astrazione della programmazione orientata agli oggetti).

- **Complessità Spaziale:** $O(N)$
- **Ricerca (get / contains):** $O(\log N)$
- **Inserimento (put):** $O(\log N)$
- **Stampa di un intervallo (range):** $O(\log N + K)$, dove K è il numero di nodi all'interno dell'intervallo richiesto.
- **Stampa completa (print):** $O(N)$

Node

Rappresenta l'unità atomica dell'albero. Ogni nodo incapsula la logica di connessione topologica e il payload informativo. Contiene i puntatori ai figli (left, right), la chiave, il valore e un booleano color (RED o BLACK) che definisce il colore del link entrante dal nodo padre. È una classe private accessibile solo dall'interno della classe RedBlackBST.

- **Complessità Spaziale per singolo nodo:** $O(1)$
- **Accesso ai figli / Valutazione colore:** $O(1)$

Book

Struttura dati utilizzata come *Value* associato alla chiave. Incapsula i campi titolo, autore e anno.

- **Complessità Spaziale:** $O(1)$

StringBuilder (Supporto I/O)

Utilizzato per accumulare stringhe durante l'attraversamento in-order dell'albero (funzioni PRINT e RANGE) prima di inviarle al buffer di stampa.

- **Append:** Complessità ammortizzata $O(1)$

2.4 Descrizione dell'algoritmo

Fase 1: Parsing e Costruzione Iniziale

Il file di input viene scansionato sequenzialmente. Il parser estrapola il comando (es. INSERT), estrae l'ISBN come Chiave (Stringa) e assembla un oggetto Book come Valore. Questi dati vengono passati alla radice dell'albero. Se l'albero è vuoto, il primo nodo diventa la radice e il suo colore viene forzato a NERO per definizione strutturale.

Fase 2: Ricerca e Aggiornamento (Dinamica Top-Down)

Per ogni chiave k da elaborare, l'algoritmo confronta k con la chiave del nodo corrente utilizzando il metodo lessicografico **compareTo()**:

- Se $k < \text{nodo.key}$, scende nel sottoalbero sinistro.
- Se $k > \text{nodo.key}$, scende nel sottoalbero destro.
- Se $k == \text{nodo.key}$, il record esiste (hit). Nel caso di un INSERT, il payload Book viene aggiornato in tempo $O(1)$. Nel caso di una SEARCH, il valore viene restituito all'utente.

Fase 3: Inserimento e Bilanciamento (Dinamica Bottom-Up)

Se la ricerca della Fase 2 termina in un *link null* (miss), viene istanziato un nuovo Node con link entrante RED. A questo punto, mentre lo stack di ricorsione risale verso la radice, l'algoritmo ripristina le regole strutturali dell'L-LRB eseguendo tre possibili operazioni di ribilanciamento:

1. **Rotate Left:** Se il figlio destro è ROSSO e il sinistro è NERO, l'algoritmo esegue una rotazione a sinistra per forzare il link rosso a pendere da quel lato.

2. **Rotate Right:** Se il figlio sinistro è ROSSO e anche il *nipote* sinistro è ROSSO (violazione del limite di link rossi consecutivi), l'algoritmo esegue una rotazione a destra per bilanciare i pesi.
3. **Flip Colors:** Se entrambi i figli di un nodo risultano ROSSI, l'algoritmo inverte i colori: i figli diventano NERI e il nodo padre diventa ROSSO, propagando il rosso (e quindi il bilanciamento) verso l'alto nell'albero.

Fase 4: Query di Range e comando Print

Alla ricezione del comando **RANGE(k1, k2)**, l'algoritmo esegue un attraversamento in-order parziale, ottimizzato tramite un processo di *pruning*:

1. Esamina il sottoalbero sinistro solo se la chiave corrente è maggiore del limite inferiore $k1$.
2. Valuta il nodo corrente: se la chiave è compresa tra $k1$ e $k2$, estrae il valore e lo appende all'output.
3. Esamina il sottoalbero destro solo se la chiave corrente è minore del limite superiore $k2$.

Questo approccio garantisce l'estrazione dei dati in perfetto ordine alfabetico senza la necessità di esplorare rami fuori dall'intervallo richiesto.

Nel caso della ricezione di **PRINT** l'algoritmo esegue una visita in-order di tutto l'albero.

2.4.1 Dettaglio implementativo

1. Operazioni atomiche di Ribilanciamento

Le tre operazioni seguenti vengono richiamate durante la fase di risalita dello stack ricorsivo per ripristinare le regole strutturali dell'albero (vincolo Left-Leaning e assenza di link rossi consecutivi).

// Rotazione a Sinistra: corregge i link rossi pendenti a destra

```
private Node rotateLeft(Node h) {  
    Node x = h.right;  
    h.right = x.left;  
    x.left = h;  
    x.color = h.color;  
    h.color = RED; // Il nodo originale diventa rosso per propagazione  
    return x;  
}
```

// Rotazione a Destra: corregge le violazioni di link rossi consecutivi

```
private Node rotateRight(Node h) {  
    Node x = h.left;  
    h.left = x.right;  
    x.right = h;  
    x.color = h.color;  
    h.color = RED;
```

```

        return x;
    }

// Inversione dei Colori: propaga il bilanciamento verso la radice
private void flipColors(Node h) {
    h.color = !h.color;
    h.left.color = !h.left.color;
    h.right.color = !h.right.color;
}

```

2. La complessità dell'operazione di Cancellazione (DELETE)

L'operazione di cancellazione fisica di un nodo in un L-LRB è considerata una delle implementazioni più ostiche nell'ambito delle strutture dati. A differenza dell'inserimento, che gestisce le violazioni dal basso verso l'alto, la delete richiede di spingere proattivamente un "link rosso" temporaneo dall'alto verso il basso (tramite complesse routine di supporto come `moveRedLeft` e `moveRedRight`) per garantire che il nodo da eliminare non sia mai una foglia nera, il che violerebbe l'invariante dell'altezza nera.

```

private Node delete(Node h, String key) {
    // Ricerca nel sottoalbero sinistro
    if (key.compareTo(h.key) < 0) {
        // Spinge un link rosso a sinistra se necessario
        if (!isRed(h.left) && !isRed(h.left.left)) {
            h = moveRedLeft(h);
        }
        h.left = delete(h.left, key);
    }
    // Ricerca nel sottoalbero destro o nodo trovato
    else {
        if (isRed(h.left)) {
            h = rotateRight(h);
        }
        // Nodo foglia trovato: cancellazione diretta
        if (key.compareTo(h.key) == 0 && (h.right == null)) {
            return null;
        }
        // Spinge un link rosso a destra se necessario
        if (!isRed(h.right) && !isRed(h.right.left)) {
            h = moveRedRight(h);
        }
    }
}

```

```

// Nodo interno trovato: sostituzione con il successore
if (key.compareTo(h.key) == 0) {
    Node x = min(h.right); // Trova il minimo nel ramo destro
    h.key = x.key;
    h.val = x.val;
    h.right = deleteMin(h.right);
} else {
    h.right = delete(h.right, key);
}
}

// Ripristino degli invarianti L-LRB in fase di risalita
return balance(h);
}

// Routine di bilanciamento universale
private Node balance(Node h) {
    if (isRed(h.right)) h = rotateLeft(h);
    if (isRed(h.left) && isRed(h.left.left)) h = rotateRight(h);
    if (isRed(h.left) && isRed(h.right)) flipColors(h);
    return h;
}

```

2.5 Architettura software

Il sistema è stato ingegnerizzato seguendo il principio cardine della **Separazione delle Competenze**, al fine di garantire un basso accoppiamento e un'alta coesione tra i moduli. L'architettura è modulare, sviluppata nativamente in Java senza l'ausilio di framework esterni, ed è organizzata nei seguenti livelli logici:

- **Livello Dati:**
 - ***Book.java***: Funge da contenitore per i campi informativi associati ad un record del dizionario ed espone i metodi per l'accesso ai campi in sola lettura, garantendo l'incapsulamento del record informativo indipendentemente dall'algoritmo che lo gestisce.
- **Livello Algoritmico:**
 - ***RedBlackBST.java***: È il motore matematico dell'applicazione. Contiene la classe privata interna *Node* (che definisce la topologia) e implementa la logica algoritmica di bilanciamento e le operazioni del dizionario, oltre alla query range e al comando print.
- **Livello di Orchestrazione e I/O:**

- **Main.java:** Funge da punto d'accesso al programma. Si occupa dell'interfacciamento con il sistema operativo, del *parsing* sequenziale del file di testo in ingresso, dell'istanziamento dell'Albero e dell'indirizzamento dell'output formattato su file. Traccia, inoltre, le metriche di esecuzione (Nodi, Altezza, Tempo) tramite le API di diagnostica di Java (*System.nanoTime()*).
- **Suite di Automazione e Testing:**
 - **GeneratoreTest.java:** Un modulo ausiliario indipendente progettato per generare dataset sintetici massivi. Supporta l'iniezione di parametri da riga di comando per pilotare dinamicamente l'attivazione o disattivazione dei carichi di I/O, permettendo la netta separazione tra test di validazione logica e test di *benchmarking* prestazionale.

Il ciclo di vita dell'applicazione prevede il caricamento istantaneo in RAM del blocco algoritmico, seguito da un *parsing* riga per riga del file in ingresso (stream processing). Questo evita di dover caricare l'intero file testuale in memoria, mantenendo un'impronta spaziale asintoticamente confinata a $O(N)$ limitata solo ai nodi effettivamente allocati nell'albero.

2.6 Miglioramenti possibili

Nonostante l'architettura implementata soddisfi rigorosamente i requisiti algoritmici, l'analisi delle performance ha evidenziato alcune aree passibili di evoluzione futura:

1. **Sostituzione con B-Tree per scalabilità out-of-core** (sotto una nuova ipotesi che non implichi l'utilizzo esclusivo di alberi binari): L'albero L-LRB è una struttura *In-Memory*, progettata per vivere interamente nello spazio di indirizzamento della RAM. Qualora il numero di record superasse le decine di milioni (causando un *OutOfMemoryError* nella JVM), il limite hardware supererebbe quello algoritmico. Il miglioramento consisterebbe nel sostituire l'RB-Tree con un B-Tree, ottimizzato appositamente per limitare le latenze del disco rigido massimizzando il numero di chiavi in un singolo nodo e riducendo i *page fault*.
2. **Separazione assoluta del Benchmarking:** Come emerso in fase di *tuning* degli script, le operazioni di stampa a schermo o su file (PRINT) introducono un collo di bottiglia I/O che inquina le metriche prestazionali della CPU. Per future misurazioni industriali, si dovrebbe implementare un framework di profilazione dedicato che aggiri totalmente le chiamate al sistema operativo.
3. **Transizione verso un'architettura puramente iterativa:** L'attuale implementazione dell'L-LRB (basata sul modello di Sedgewick) fa un uso intensivo della **ricorsione** per la discesa nell'albero e per il successivo ribilanciamento top-down/bottom-up (con le chiamate a *rotateLeft*, *rotateRight* e *flipColors* che scattano alla chiusura dei record di attivazione). Sebbene questo renda il codice estremamente elegante e matematicamente provabile, introduce un leggero *overhead* prestazionale dovuto al continuo riempimento del Call Stack della memoria. Un possibile miglioramento prestazionale a basso livello consisterebbe nel tradurre i metodi *put* e *delete* in versioni **puramente iterative**. Questo abbatterebbe i tempi costanti di esecuzione, pur sacrificando notevolmente la leggibilità e la manutenibilità del codice sorgente.
4. **Ottimizzazione della cancellazione tramite Lazy Deletion:** Durante lo sviluppo, è emerso che l'operazione di DELETE in un albero L-LRB è asimmetricamente più complessa dell'inserimento, richiedendo complesse rotazioni per propagare i link rossi verso il basso e non violare l'altezza nera. Qualora il sistema reale presentasse un pattern di accesso con

rarissime cancellazioni e frequentissime ricerche (tipico dei cataloghi storici), un netto miglioramento algoritmico consisterebbe nell'adottare la tecnica della **Lazy Deletion** (Cancellazione Pigrà). Invece di rimuovere fisicamente il nodo e ribilanciare l'albero, basterebbe aggiungere un flag booleano `isDeleted` alla classe `Node`. L'operazione di `DELETE` scenderebbe a un costo temporale di $O(1)$ post-ricerca, delegando a un processo in background (o al raggiungimento di una soglia di "rifiuti") la ricostruzione pulita dell'albero.

CAPITOLO 3: ANALISI SPERIMENTALE

3.1 Complessità

Per giustificare a livello teorico la scelta dell'Albero Rosso-Nero L-LRB, si riporta di seguito la tabella comparativa delle complessità asintotiche spaziali e temporali rispetto alle strutture dati alternative valutate in fase di analisi. La variabile N rappresenta il numero di record nel database, mentre la variabile K rappresenta il numero di elementi restituiti all'interno di un intervallo di ricerca.

Struttura Dati	Ricerca (Pessimo)	Inserimento (Pessimo)	Cancellazione (Pessimo)	Range	Spazio RAM	Costo totale su file
BST Standard	$O(N)$	$O(N)$	$O(N)$	$O(N)$	$O(N)$	$O(N^2)$
Albero AVL	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N + K)$	$O(N)$	$O(N \log N)$
L-LRB Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N + K)$	$O(N)$	$O(N \log N)$

(I valori all'interno della tabella non giustificano da soli la scelta degli L-LRB, a discapito degli AVL. Le considerazioni fatte nel Capitolo 2.2 sono essenziali per comprendere le decisioni effettuate).

3.2 Dataset e casi di test

Per convalidare la correttezza e la scalabilità dell'algoritmo, è stata sviluppata una suite di automazione basata su script (.bat / .sh) e un modulo Java generatore di *dataset* sintetici (GeneratoreTest.java).

I test sono stati condotti su file di input di taglia esponenzialmente crescente: $N \in \{10, 100, 1.000, 10.000, 100.000, 1.000.000\}$ operazioni.

Il rigore metodologico dei test si basa su due scelte architetturali fondamentali:

1. **Isolamento del rumore di sistema:** Per ogni taglia N , lo stesso identico file di input è stato eseguito per 10 *run* consecutive. Il tempo finale registrato è la media aritmetica di queste esecuzioni. Questo approccio di *System Benchmarking* permette di ammortizzare le latenze introdotte dal Sistema Operativo, dal *page caching* del disco e dai cicli del *Garbage Collector* della Java Virtual Machine (JVM). Non è stato necessario variare l'ordine dei dati per testare il "caso pessimo", poiché l'L-LRB garantisce asintoticamente le stesse prestazioni per qualsiasi permutazione dell'input.
2. **Separazione logica tra I/O e CPU:** Il generatore è stato parametrizzato tramite una flag da riga di comando. Per il test funzionale ($N=100$) la flag ***usaPrint=true*** ha abilitato le stampe

massive per validare visivamente l'ordine alfabetico. Per i test prestazionali di scalabilità da $N=10$ a $N=1.000.000$, lo script ha iniettato la flag ***usaPrint=false***. Questo ha disattivato dinamicamente il collo di bottiglia del disco rigido (operazioni di I/O lente), limitando le interrogazioni a *Range Query* di dimensione fissa ($K=50$). In questo modo, i tempi misurati riflettono la pura velocità di elaborazione della CPU nella memoria RAM, tracciando una curva fedele alla complessità algoritmica.

3.3 ambiente, macchina, librerie

Per garantire la riproducibilità dell'esperimento, si documenta l'ambiente hardware e software utilizzato per l'estrazione delle metriche:

1. Macchina di Test (Hardware)

- **Processore (CPU):** *AMD Ryzen 7 5825U*
- **Memoria (RAM):** *16 GB DDR4*
- **Archiviazione (Disco):** *WDC PC SN530 SDBPNPZ-512G-1006*
- **Sistema Operativo:** *Windows 11 Home 64-bit (per test.sh: WSL Ubuntu-24.04 LTS)*

2. Ambiente di Sviluppo e Librerie (Software)

- **Linguaggio:** *Java Standard Edition (JDK) versione 21.*
- **Tuning JVM:** Durante gli *stress test* la Virtual Machine è stata avviata con l'argomento -Xmx4G per allocare preventivamente 4 GB di spazio *Heap* e prevenire errori di *OutOfMemory*.
- **Librerie Esterne:** **Nessuna.** L'intero progetto è stato sviluppato utilizzando esclusivamente le librerie standard (*java.util.**, *java.io.**). Non è stato utilizzato alcun framework esterno per mantenere l'architettura leggera ed evidenziare la pura implementazione algoritmica.

3.4 Risultati e validazione correttezza

1. Validazione Funzionale

Prima di procedere al benchmarking prestazionale su larga scala, la correttezza semantica delle operazioni del dizionario è stata validata tramite piccoli file di testo. Di seguito si riporta il risultato di uno di questi test, che dimostra il corretto funzionamento delle operazioni di insert, search, delete, range e print.

Input.txt:

```
INSERT 97801 Algoritmi_C++ Sedgewick 2020
INSERT 97815 Database Hoffer 2016
INSERT 97807 The Java Programming Language Arnold 1958
SEARCH 97815
DELETE 97807
RANGE 97800 97820
PRINT
```

Output.txt:

```
(SEARCH)
Libro trovato:
ISBN: 97815
Titolo: Database
Autore: Hoffer
Anno: 2016
(RANGE 97800 97820)
97801 Algoritmi_C++
97815 Database
PRINT
97801 Algoritmi_C++
97815 Database
```

Output del terminale:

```
Tempo di esecuzione: 49,34 ms
Numero di nodi dell'albero: 2
Altezza nera dell'albero: 1
```

2. FASE DI BENCHMARK

Di seguito si riportano i tempi medi di esecuzione misurati in pura RAM per il *parsing* dei file e la conseguente elaborazione sull'albero L-LRB. I tempi esposti rappresentano la media aritmetica calcolata su 10 esecuzioni consecutive per ogni taglia N , al fine di isolare le latenze del Sistema Operativo.

Dimensione Dataset (N)	Nodi finali	Altezza nera finale	Tempo medio di esecuzione (ms)
10	9	3	~ 2,05
100	81	5	~ 34,54
1.000	788	8	~ 55,58
10.000	7.937	11	~ 299,85
100.000	79.671	13	~ 527,86
1.000.000	765.169	16	~ 3.565,34

Analisi delle Prestazioni Asintotiche:

I risultati sperimentali confermano l'analisi di complessità teorica. L'algoritmo si dimostra in grado di sostenere l'inizializzazione e l'elaborazione di un milione di operazioni in circa 3,5 secondi. Osservando il passaggio da 100.000 a 1.000.000 di operazioni (un incremento del dataset di un fattore 10), il tempo di esecuzione cresce in modo proporzionato, tracciando una progressione limitata dal fattore $O(N \log N)$. Questo evidenzia la buona scalabilità della struttura per database *In-Memory* su larga scala.

Validazione di Correttezza:

Oltre alla validazione funzionale dei file in output (verificata tramite il test isolato da 100 righe), la stabilità topologica dell'albero è stata dimostrata estraendo l'altezza nera della radice al termine di ogni esecuzione.

Come si evince dalla tabella, a fronte di un milione di inserimenti dinamici e casuali (con oltre 765.000 nodi unici allocati fisicamente), l'altezza nera si è assestata a **16**. Essendo il limite teorico massimo per un Albero L-LRB pari a $2 \log_2 N$, un'altezza nera di 16 conferma che la struttura non è degenerata e ha mantenuto il bilanciamento atteso, attraverso le rotazioni applicate in fase di inserimento.